HƯỚNG DẪN LÀM BÀI

Bài 1: Kế thừa cơ bản (Lớp Circle và Cylinder)

Mục tiêu bài toán: Xây dựng lớp hình trụ (Cylinder) kế thừa từ lớp hình tròn (Circle).

1. Hình trụ bản chất là một hình tròn được kéo dài thêm chiều cao (height). Do đó, Cylinder extends Circle.
2. Thuộc tính radius và color ở lớp Circle đang để private, lớp con không thể truy cập trực tiếp mà phải thông qua get/set hoặc super().

Khi tính thể tích hình trụ NÊN tái sử dụng phương thức getArea() của lớp cha.

```java
public class Cylinder extends Circle {
    private double height = 1.0;

    // Gọi constructor của lớp cha bằng từ khóa super
    public Cylinder(double radius, double height, String color) {
        super(radius, color);
        this.height = height;
    }

    // Tái sử dụng getArea() của lớp Circle để tính thể tích
    public double getVolume() {
        return super.getArea() * this.height;
    }

    @Override
    public String toString() {
        return "Cylinder[" + super.toString() + ", height=" + height + "]";
    }
}
```

Bài 2: Hệ thống phân cấp Person - Student - Staff

Mục tiêu bài toán: Từ một lớp gốc Person, xây dựng hai lớp cụ thể hơn là Student (Sinh viên) và Staff (Nhân viên).

- Constructor của lớp con bắt buộc phải khởi tạo các thuộc tính của lớp cha trước tiên thông qua super(name, address).
- Cách viết hàm toString() chuỗi kết hợp.

GỢI Ý:

```java
public class Student extends Person {
    private String program;
    private int year;
    private double fee;

    public Student(String name,
                   String address,
                   String program,
                   int year,
                   double fee) {
        super(name, address);
        this.program = program;
        this.year = year;
        this.fee = fee;
    }

    @Override
    public String toString() {
        return "Student[" + super.toString() +
               ", program=" + program +
               ", year=" + year +
               ", fee=" + fee + "]";
    }
}
```

Bài 3: Lớp Point2D và Point3D

Mục tiêu bài toán: Xây dựng điểm trong không gian 2 chiều (Point2D) và mở rộng lên 3 chiều (Point3D).

- Trọng tâm bài này là làm quen với việc trả về và nhận vào một **Mảng (Array)** kiểu float[] trong các phương thức getXY() và setXYZ().

Cách khởi tạo mảng ẩn danh (anonymous array) ngay trong lệnh return.

```
// Trong lớp Point2D
public float[] getXY() {
    return new float[]{this.x, this.y}; // Trả về mảng 2 phần tử
}

// Trong lớp Point3D
public void setXYZ(float x, float y, float z) {
    super.setXY(x, y); // Gọi hàm của Point2D để set x và y
    this.z = z;
}

public float[] getXYZ() {
    return new float[]{super.getX(), super.getY(), this.z};
}
```

Bài 4: Hệ thống phân cấp Shape (Bài tập kinh điển)

Mục tiêu bài toán: Thiết kế chuỗi kế thừa đa cấp: Shape => Rectangle => Square (và Circle kế thừa trực tiếp Shape).

- Một hình vuông (Square) là một hình chữ nhật (Rectangle) có 2 cạnh bằng nhau. Do đó, khi thay đổi chiều rộng của hình vuông, chiều dài cũng **bắt buộc** phải thay đổi theo để giữ tính chất hình vuông.

GỢI Ý:

```java
public class Square extends Rectangle {

    public Square(double side) {
        super(side, side); // Truyền side cho cả width và length của Rectangle
    }

    @Override
    public void setWidth(double width) {
        super.setWidth(width);
        super.setLength(width); // Chiều dài và rộng phải thay đổi cùng nhau
    }

    @Override
    public void setLength(double length) {
        super.setLength(length);
        super.setWidth(length);
    }
}
```

Bài 5: Quản lý Nhân viên (Employee) và Logic Nghiệp vụ

Mục tiêu bài toán: Kết hợp các thuộc tính để giải quyết một bài toán tính lương thực tế có điều kiện (if/else).

- Bài này không có kế thừa, nhưng tập trung vào logic tính toán bên trong các phương thức (Business logic).
- Cần xử lý linh hoạt việc tính năm làm việc (có thể ấn định năm hiện tại hoặc dùng thư viện ngày tháng của Java).

GỢI Ý:

```java
public class Employee {
    // ... các thuộc tính ...

    // Tính lương thâm niên
    public double getSenioritySalary() {
        // Hoặc dùng Year.now().getValue()
        int currentYear = 2026;
        int yearsOfWork = currentYear - this.yearJoined;

        if (yearsOfWork >= 5) {
            double basicSalary = 1150;
            return (yearsOfWork * basicSalary) / 100.0;
        }
        return 0;
    }
```

```java
    // Xét thi đua
    public String considerEmulation() {
        if (this.numDaysOff <= 1) {
            return "A";
        // Giả định <= 3 là B dựa trên logic phân loại
        } else if (this.numDaysOff <= 3) {
            return "B";
        } else {
            return "C";
        }
    }

    // Tính tổng lương
    public double getSalary() {
        double basicSalary = 1150;
        double emulCoeff = 1.0;

        String rank = considerEmulation();
        switch (rank) {
            case "A": emulCoeff = 1.0; break;
            case "B": emulCoeff = 0.75; break;
            case "C": emulCoeff = 0.5; break;
        }

        // basic salary * salary coefficient * emulation + seniority
        return (basicSalary * this.coefficientsSalary * emulCoeff)
               + getSenioritySalary();
    }
}
```